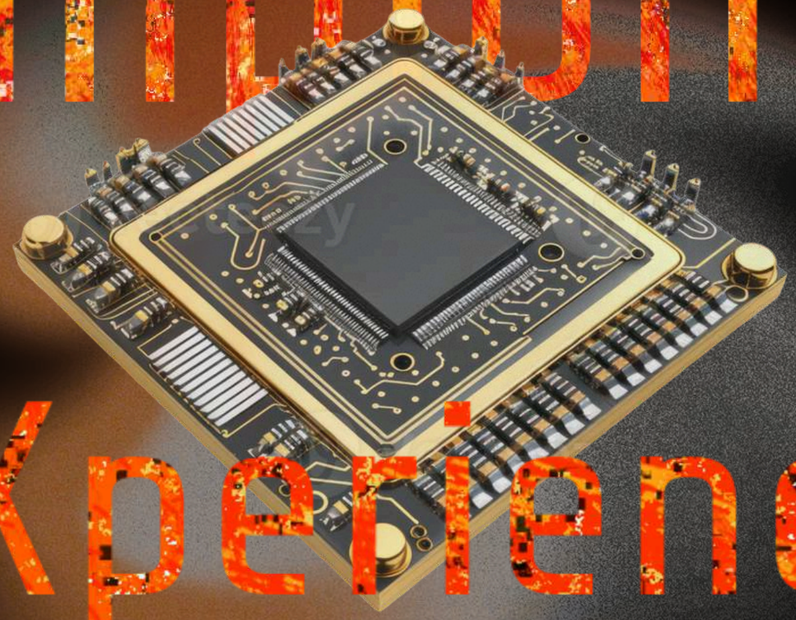
ChampionCHIP eXperience

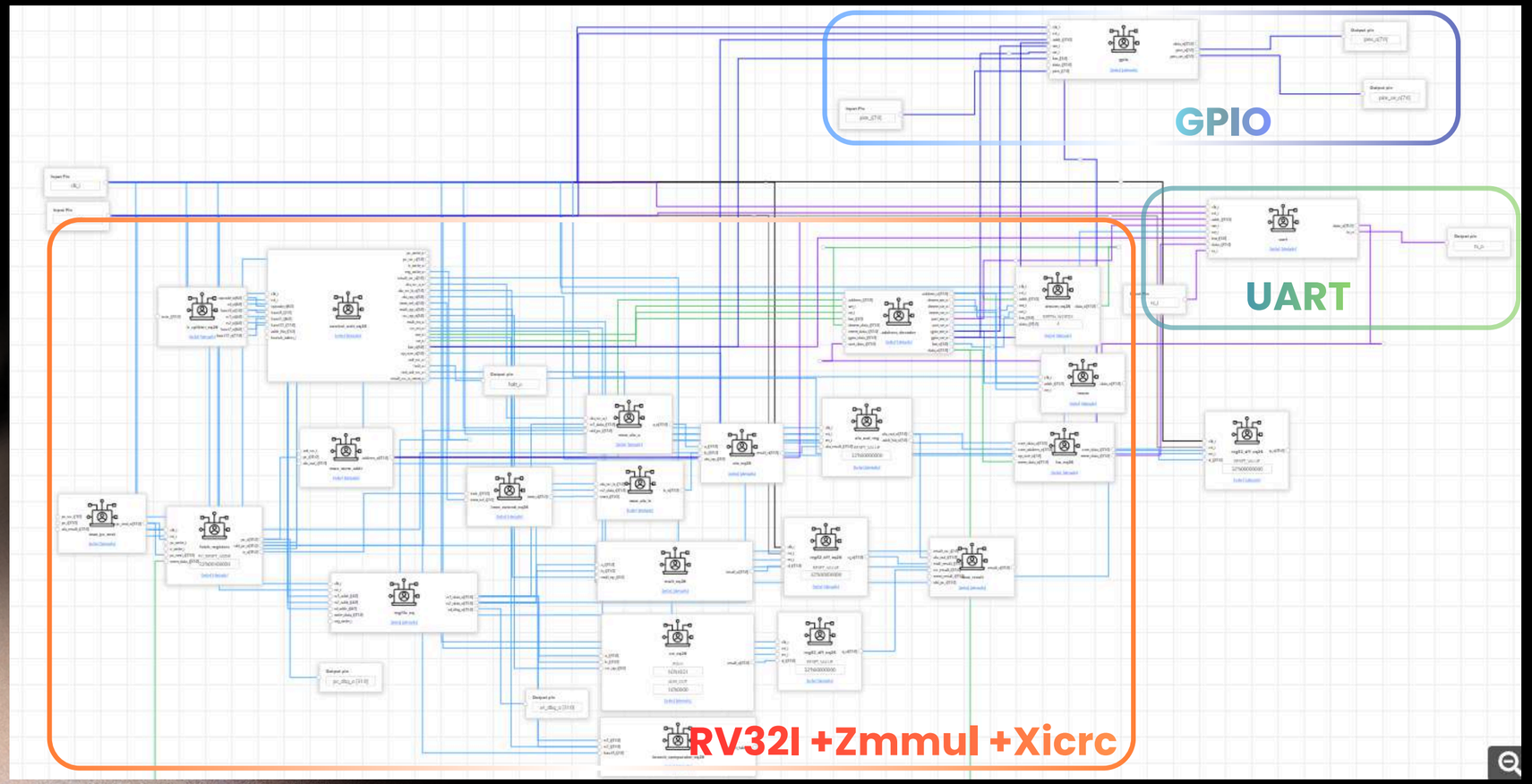


Grid-aware EV charging controller on the RVBL-2 RISC-V core

Equipe 26

CHUA ZHU HENG, LEE SHI YI, TAN DE MING, SHAWN BOO

PROJECT ARCHITECTURE



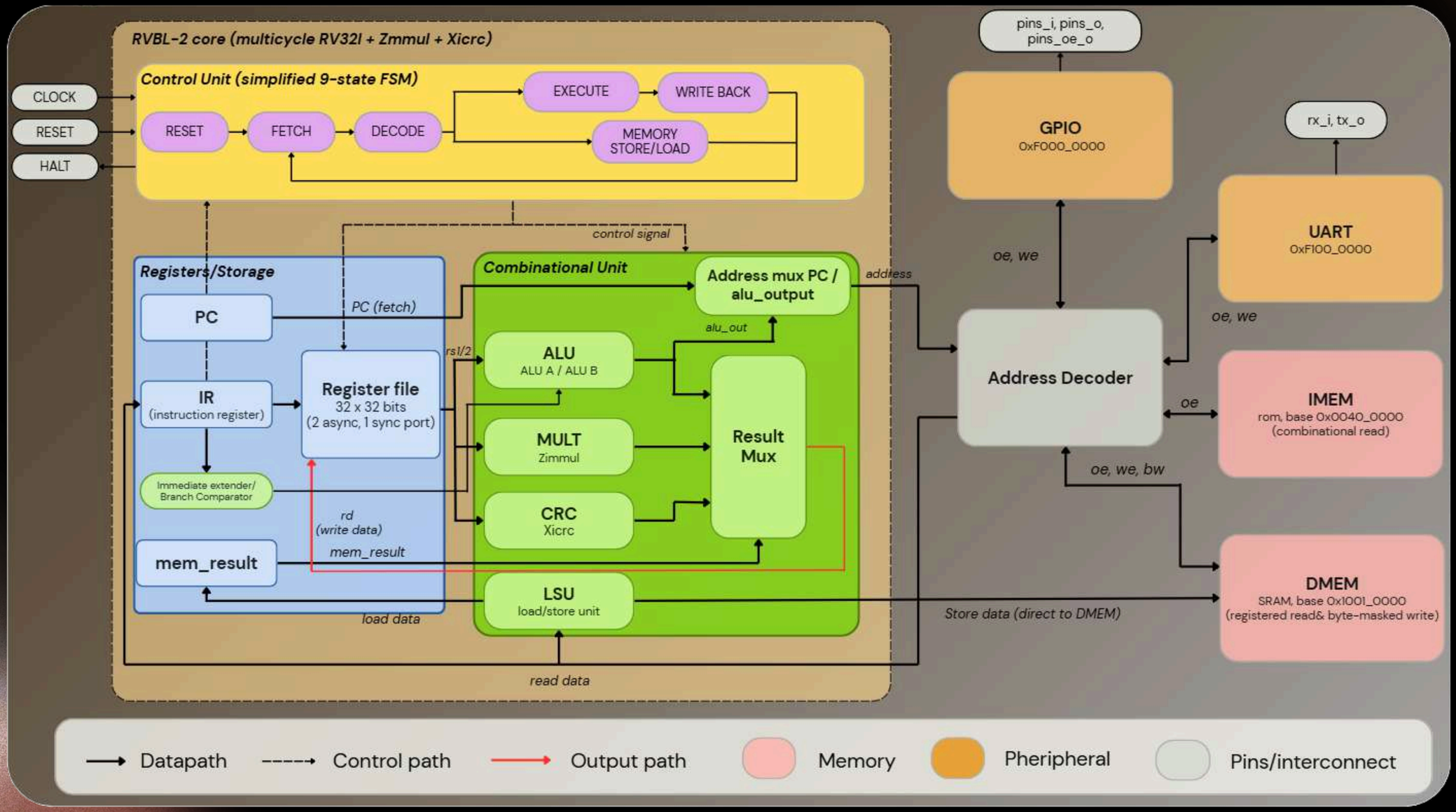
One chip, everything a charger needs

- *RVBL-2 multicycle RISC-V core*
- *RV32I + Zmmul + Xicrc (custom CRC extension)*
- *GPIO: 8 pins*
- *UART: 115,200 baud, 8N1*

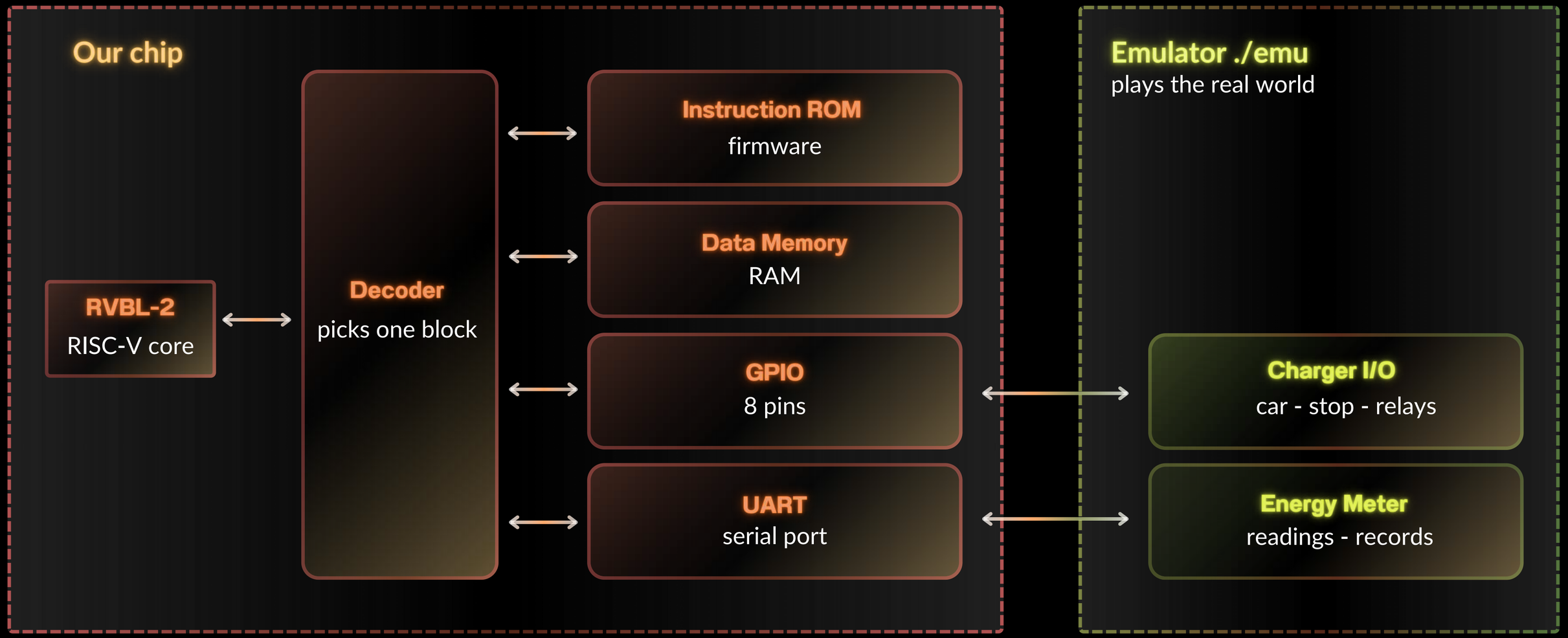
Instruction ROM · 32-byte data memory · address decoder

Core, memory and both peripherals on one die.

PROJECT ARCHITECTURE



HARDWARE DESIGN



Core, memory and both peripherals on one die.

FIRMWARE IMPLEMENTATION

Feed inputs in, let the chip process, check expected outputs.

Read Meter Data
UART

INPUTS

- Pilot signal from car (plugged in, wants charge, error)
- Emergency stop button
- Meter reading over UART: 8 bytes per frame (voltage, current, frequency, checksum)

```
Frame:  230V, 16A, 50.00Hz, CRC
```

Check & Calculate
CRC + MULTIPLY

FIRMWARE DECISIONS

- Verify checksum with crcb — bad frame? Throw it away
- Compute energy: voltage × current, added to running total
- Watch grid health: frequency below 49.50 Hz or voltage too low? Hold charging
- Detect faults: E-stop, over-current, two bad frames in a row, invalid pilot

```
crcb  a4, a4, a0   # verify m01
t0, a5, a5   # energy
```

Control Charger
GPIO

EXPECTED OUTPUTS

- Lock plug → keep cable connected
- Switch power on or off (contactor)
- Hold charging during a grid dip
- Signal fault and stop everything

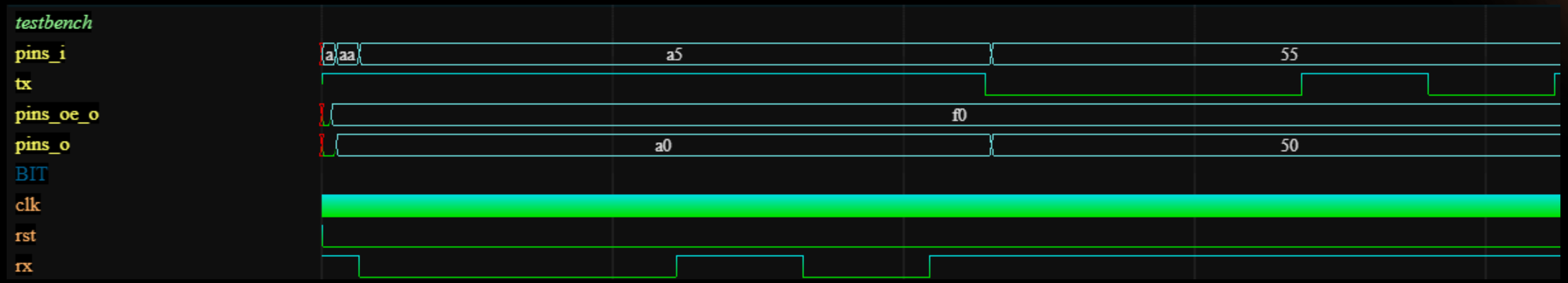
```
0x20 lock 0x30 charge 0x50 grid
hold 0xE0 fault
```

APPLICATION STATES (VISIBLE ON DEBUG REGISTER X4)

IDLE 0x00 → CONNECTED 0x20 → CHARGING 0x50 → GRID HOLD 0x80 → FAULT 0xE0

- ✓ **Two FSMs** — hardware executes, firmware decides
- ✓ **No RAM** — all state in registers, fits cheapest chips
- ✓ **Same firmware** runs in simulation, FPGA, and silicon

OFFICIAL FIRMWARE TESTBENCH



Simulation Log

```
VCD info: dumpfile testbench/testbench.vcd opened for output.
PASS [GPIO] P3-0 = 0xa, P7-4 = 0xa (0 clocks)
PASS [UART] RX = 0x30, TX = 0x30
PASS [GPIO after UART] P3-0 = 0x5, P7-4 = 0x5 (0 clocks)
PASS: GPIO and UART firmware test completed successfully
testbench/testbench.v:143: $finish called at 173825000 (1ps)
```

GPIO and UART are successfully integrated and validated using the official firmware testbench.

PROBLEM STATEMENT

1. Dip
A power station trips



1,000 Chargers **7kW** Each

50Hz Normal grid **7pm** peak demands

Supply < Demand ⟶ Freq Drop

- Grid frequency falls below 50 Hz
- Every charger stops to protect the grid

Stopping is the safe move

2. Recover
Everyone restarts at once



Grid OK → All restart → Same instant

- Every charger sees the same recovery signal
- No charger knows what the others will do
- Fixed delays don't help: everyone still waits the same time

Restarting together is the danger

3. Rebound
The grid trips again



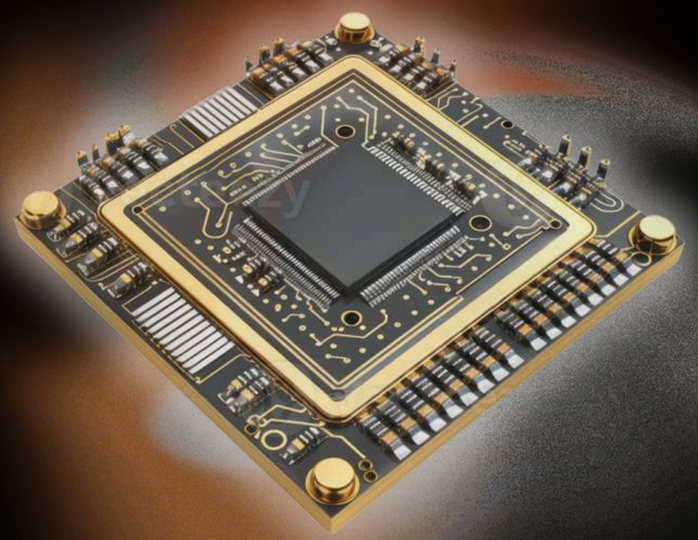
7MW in one instant

- 1,000 × 7 kW returns in the same second
- A weak, just-recovered grid dips again
- The cycle repeats

One dip becomes many

Chargers protect the grid when it dips, then trip it again when they all come back.

PROPOSED SOLUTION



Gripple

RVBL-2 RISC-V core + custom crcb instruction

Integrity-checked metering

*Every **meter reading is CRC-checked** and chained to the one before it. Corrupted, dropped and replayed readings are **never billed**.*

Staggered restart

*Each charger restarts at its own moment, computed from its ID and the grid event. **Different chargers get different delays**, so 1000 restart spread across 8 time slots instead of one instant.*

No network, no clock, no random numbers

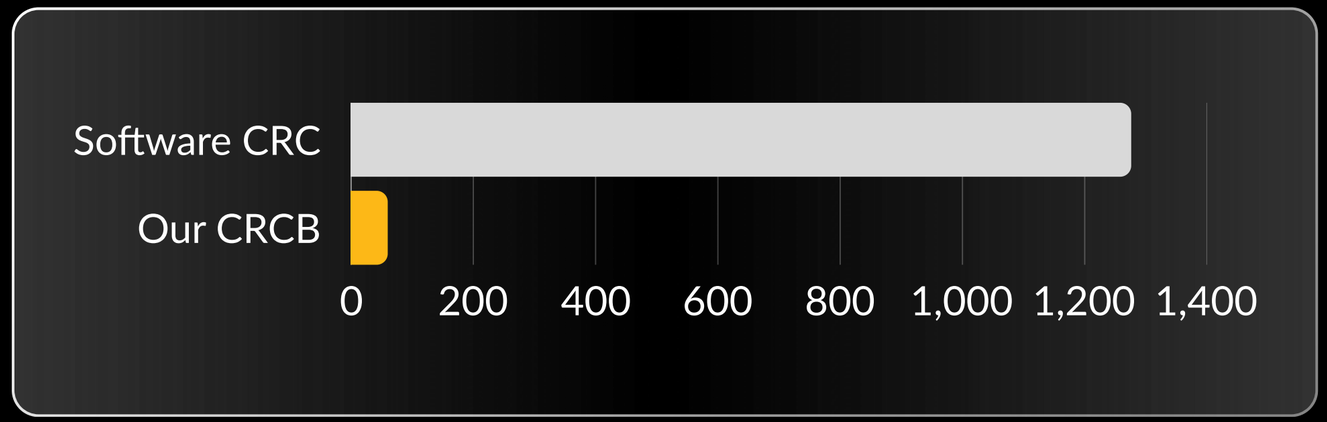
*The **delay is calculated on the chip**, from data every charger already has. It keeps working when communications fail, which is often during the same grid event.*

One instruction, three jobs

*Our custom instruction crcb checks each reading, chains readings together, and computes the restart delay. **21× faster than doing CRC in software**.*

CUSTOM INSTRUCTION (crcb)

Cycles to CRC one meter frame
(6 bytes, measured)



21X ***fewer clock cycles to compute the CRC-16 of one meter frame***



Hardware does all 8 bit-steps at once, so a CRC is as fast as an add

Same instruction format as add / mul · crcb = 1 byte · crch = 2 bytes · crcw = 4 bytes

1 CHECK — ***Every reading is checked on arrival***

*Damaged readings are never trusted or billed, and checking uses just **0.2% of the chip's time**.*

2 CHAIN — ***Every reading is linked to the last one***

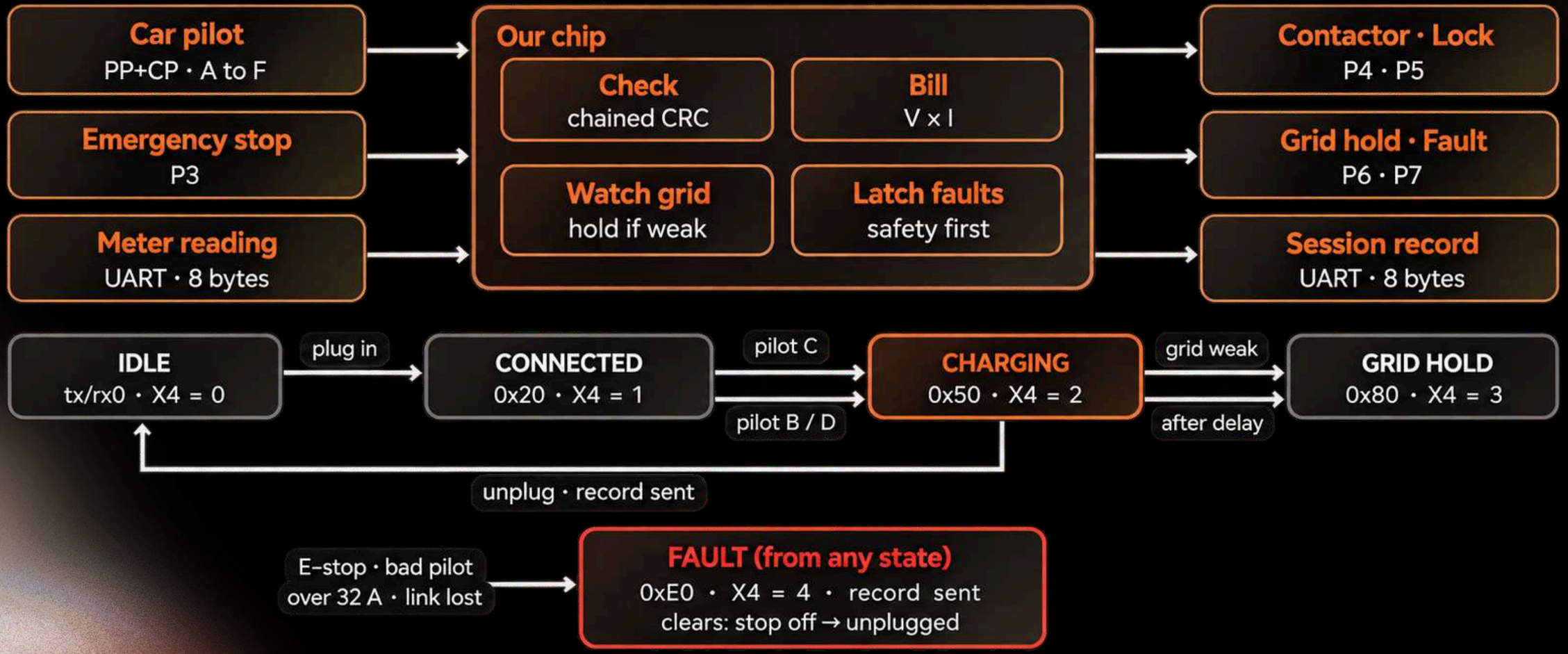
***Missing or repeated readings are caught** without sending any extra data.*

3 TIME — ***Every charger gets its own restart moment***

*Chargers **restart in turns** instead of all at once, cutting the restart surge nearly 8× (7 MW → 0.88 MW) so the **grid isn't knocked down** a second time.*

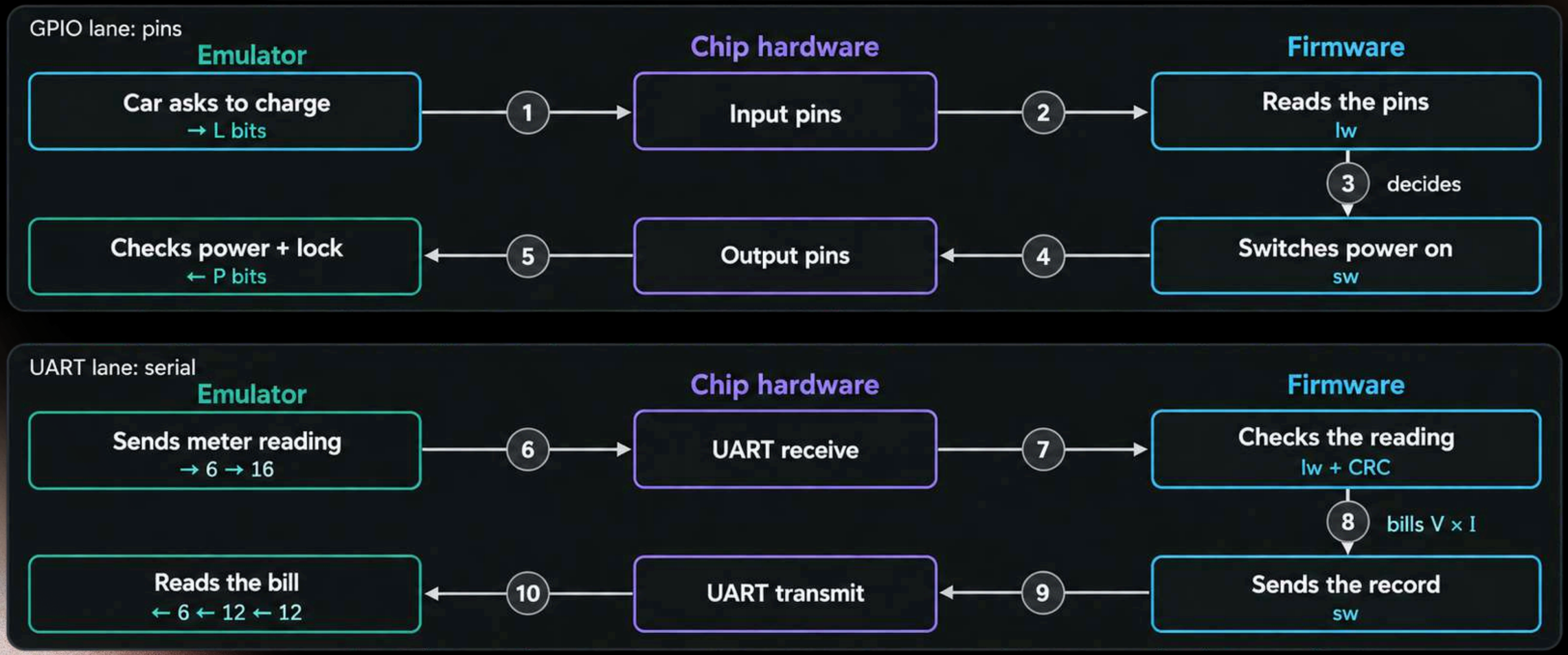
SYSTEM OVERVIEW

How our chip controls a public AC EV charging station



APPLICATION LOGIC

The emulator plays the car and the meter; the chip answers through its pins and serial port



The chip only uses load and store: no special hooks, so the same firmware runs on the FPGA unchanged.

EMULATOR

2 · Application Logic

sends: emulator → chip checks: chip → emulator

```
./emv -d 0xF0 -l 0x00 -o 0x00 -t4 0x01 -o 0x20 -4 0x02 -o 0x30
-t4 0x00400BFC -t4 0xC7771338 -o 0x30 -t4 0x00A40BFC -t4 0xDD0E51338 -o 0x30   same reading, new CRC, chained
-t4 0x01 -o 0x20 -t4 0x00 -o 0x00 -r4 0x00007360 -r2 0x0200 -r2 0x005C
```

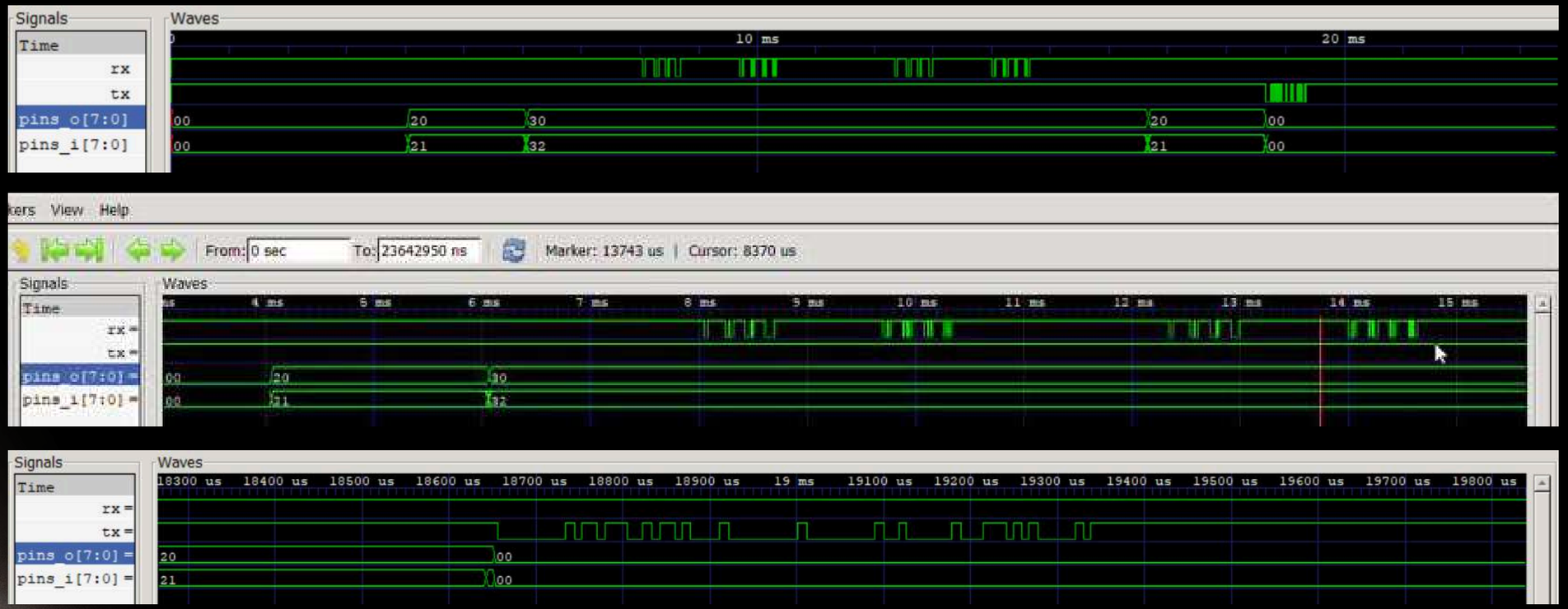
OPTION	STEP	WHAT IT MEANS
`-d 0xF0`	setup	P7–P4 chip outputs · P3–P0 driven by the emulator
`-t4 0x01`	①	Car plugged in (pilot B)
`-o 0x20`	②	Expect: lock closed
`-t4 0x02`	③	Car asks to charge (pilot C)
`-o 0x30`	④	Expect: lock + contactor closed
`-t4 0x00400BFC`	⑤	Meter: 230.0 V, 16.0 A
`-t4 0xC7771338`	⑥	Meter: 50.00 Hz, CRC 0xC777
`-t4 0x00A408FC`	⑦	Second reading: same values, new CRC 0xD0E5 (chained), then unplug
`-r4 0x00007360`	⑧	Record: energy 736,000
`-r2 0x0200`	⑨	Record: 2 readings billed, no faults
`-r2 0x005C`	⑩	Record: its own CRC

`-t4` sends 4 bytes, low byte first · **`-r2`** compares pairs of received bytes

DEMO 1: NORMAL CHARGING SESSION

```
PS C:\Users\zhchua\Documents\Project (Comp)\ChampionCHIP-Multicycle-Processor-Architecture> python chip-inventor/testbench/app/run_tests.py --scenario S1 --top rtl
/top.v
PASS rtl_top S1: PASS (normal session: plug in, charge two frames, unplug)

1/1 runs as expected
```

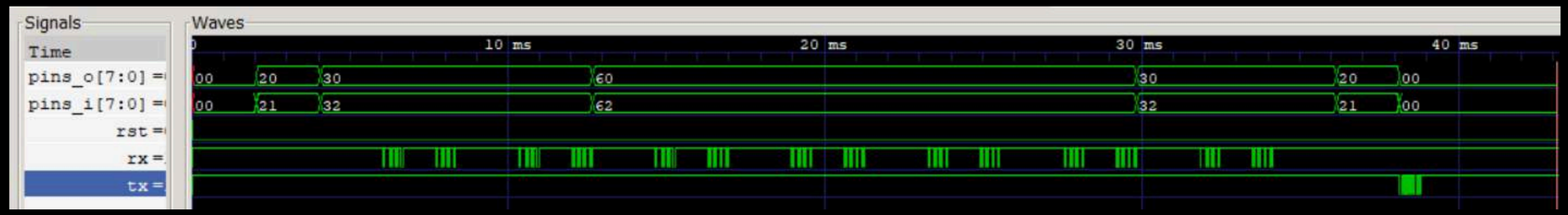


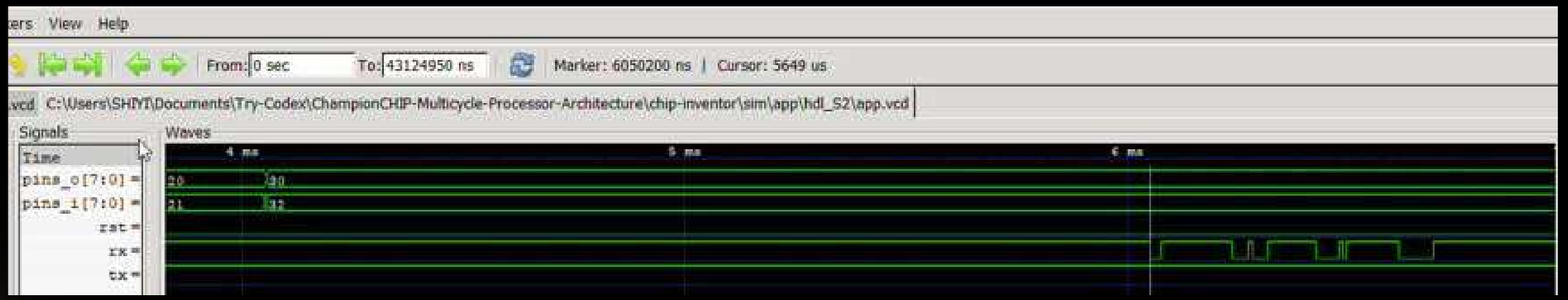
Core, memory and both peripherals on one die.

DEMO 2: GRID DIP & STAGGERED RESTART

```
PS C:\Users\zhchua\Documents\Project (Comp)\ChampionCHIP-Multicycle-Processor-Architecture> python chip-inventor/testbench/app/run_tests.py --scenario S2 --top rtl
/top.v
PASS rtl_top S2: PASS (grid under-frequency: hold, then CRC-timed restart)

1/1 runs as expected
```





Core, memory and both peripherals on one die.

VALIDATION: APPLICATION TESTBENCH

```
>> C:\Users\zhchua\Documents\Project (Comp)\ChampionCHIP-Multicycle-Processor-Architecture>
PASS hdl H0: PASS (organiser echo via emu model)
PASS hdl H1: expected-FAIL (harness catches a mismatch)
PASS hdl H2: PASS (organiser README example, verbatim (needs IN=0x02 at boot))
PASS hdl S1: PASS (normal session: plug in, charge two frames, unplug)
PASS hdl S1X: PASS (stress: S1 with back-to-back UART bytes)
PASS hdl S2: PASS (grid under-frequency: hold, then CRC-timed restart)
PASS hdl S3: PASS (corrupted frame and dropped frame each cost one frame)
PASS hdl S4: PASS (meter link lost: two bad frames in a row latch FAULT)
PASS hdl S5: PASS (E-stop while charging latches FAULT until released and unplugged)
PASS hdl S6: PASS (plug straight into CP C, then unplug while charging)
PASS hdl S7: PASS (frame exactly on every limit is billed; I_MAX+1 faults)
PASS hdl S8: PASS (ventilation (CP D) refused; CP E latches FAULT)
PASS hdl S9: PASS (invalid pilot code 7 latches FAULT with the CP-error flag)
PASS hdl S10: PASS (replayed frame rejected, and the next one too -> FAULT)
PASS hdl S11: PASS (stale pins at boot: CP C already asserted, settles to CHARGING)
PASS hdl_ref H0: PASS (organiser echo via emu model)
PASS hdl_ref H1: expected-FAIL (harness catches a mismatch)
PASS hdl_ref H2: PASS (organiser README example, verbatim (needs IN=0x02 at boot))
PASS hdl_ref S1: PASS (normal session: plug in, charge two frames, unplug)
PASS hdl_ref S1X: PASS (stress: S1 with back-to-back UART bytes)
PASS hdl_ref S2: PASS (grid under-frequency: hold, then CRC-timed restart)
PASS hdl_ref S3: PASS (corrupted frame and dropped frame each cost one frame)
PASS hdl_ref S4: PASS (meter link lost: two bad frames in a row latch FAULT)
PASS hdl_ref S5: PASS (E-stop while charging latches FAULT until released and unplugged)
PASS hdl_ref S6: PASS (plug straight into CP C, then unplug while charging)
PASS hdl_ref S7: PASS (frame exactly on every limit is billed; I_MAX+1 faults)
PASS hdl_ref S8: PASS (ventilation (CP D) refused; CP E latches FAULT)
PASS hdl_ref S9: PASS (invalid pilot code 7 latches FAULT with the CP-error flag)
PASS hdl_ref S10: PASS (replayed frame rejected, and the next one too -> FAULT)
PASS hdl_ref S11: PASS (stale pins at boot: CP C already asserted, settles to CHARGING)
PASS rtl_top H0: PASS (organiser echo via emu model)
PASS rtl_top H1: expected-FAIL (harness catches a mismatch)
PASS rtl_top H2: PASS (organiser README example, verbatim (needs IN=0x02 at boot))
PASS rtl_top S1: PASS (normal session: plug in, charge two frames, unplug)
PASS rtl_top S1X: PASS (stress: S1 with back-to-back UART bytes)
PASS rtl_top S2: PASS (grid under-frequency: hold, then CRC-timed restart)
PASS rtl_top S3: PASS (corrupted frame and dropped frame each cost one frame)
PASS rtl_top S4: PASS (meter link lost: two bad frames in a row latch FAULT)
PASS rtl_top S5: PASS (E-stop while charging latches FAULT until released and unplugged)
PASS rtl_top S6: PASS (plug straight into CP C, then unplug while charging)
PASS rtl_top S7: PASS (frame exactly on every limit is billed; I_MAX+1 faults)
PASS rtl_top S8: PASS (ventilation (CP D) refused; CP E latches FAULT)
PASS rtl_top S9: PASS (invalid pilot code 7 latches FAULT with the CP-error flag)
PASS rtl_top S10: PASS (replayed frame rejected, and the next one too -> FAULT)
PASS rtl_top S11: PASS (stale pins at boot: CP C already asserted, settles to CHARGING)

45/45 runs as expected
```

45/45 — SIMULATION RUNS

38/38 — PYTHON MODEL TEST

PASS — OFFICIAL TEST

FAULT INJECTED	RESULT	STATUS
Corrupted meter frame	Not billed — Thrown away, charging continues	PASS
Dropped frame	Not billed — Skipped, charging continues	PASS
Two bad frames in a row	FAULT — Link-lost flag	PASS
Replayed old frame	FAULT — Chained CRC catches replay	PASS
Emergency stop pressed	FAULT — Latched until released and unplugged	PASS
Over-current (32.1 A)	FAULT — 32.0 A still billed correctly	PASS
Invalid pilot signal	FAULT — Pilot-error flag set	PASS
Stale pins at power-up	Handled — Settles to correct state	PASS
Back-to-back UART bytes	Handled — No bytes lost	PASS

COMPETITIVE ANALYSIS

	Basic wallbox	Timer charger	Smart charger	Networked charger	Ours
How it restarts	at once	after a fixed or scheduled delay	after a random delay	when the back-end server says	**at its own CRC-timed moment**
Spreads the restart surge	✗	✗	✓	✓	✓
Senses grid trouble itself (frequency, voltage)	✗	✗	◐	◐	✓
Still protects the grid when the network is down	—	—	✓	✗	✓
No random-number source needed	✓	✓	✗	✓	✓
Restart moment can be re-checked afterwards	✓	✓	✗	◐	✓
Extra parts needed	none	real-time clock	random source	modem + cloud server	**none: one small chip**

✓ yes · ✗ no · ◐ some models, or only via the operator · — no grid protection to lose

Typical behaviour of each product type; individual models vary.
Only ours ticks every box.

VALUE & PRACTICAL FUNCTIONALITY

A grid being protected

8x *smaller power surges*

After a power dip, chargers switch back on in turns, not all at once.

No hassle for drivers

Charge *restarts by itself*

The car stays plugged in and locked, with no need to replug.

Every bill can be trusted

Verified *real reading are billed*

Broken, missing or repeated meter readings are never charged.

WHO	WHAT CHANGES FOR THEM	HOW OUR CHIP DELIVERS IT
Grid operator	Lower risk of a second blackout after a dip, and room for more chargers on the same grid	Staggered restarts; works with no network
Driver	No replugging after a grid hiccup, and never charged for a reading that didn't happen	Car stays locked and resumes automatically; only verified readings are billed
Charge-point operator	Fewer grounds for billing disputes and refunds	Uses its existing meter; the session record leaves sealed
Charger maker works offline	Grid support even in cheap chargers with no internet connection	No clock, random-number chip or modem; one small chip
Regulator	Rules that can be enforced after the event	Any charger's delay can be recomputed from its ID and the event

STAGE 4 FPGA ROADMAP

ASEM

ChipInventor™

1 Ready now

What we do

RTL **passes the official test** and all **12 of our scenarios**; each scenario exported as a real ./emu command

Done when

45 / 45 simulation runs pass

2 Stage 4: FPGA

What we do

Run the chip **on the AWS FPGA** against the **real emulator** using run_scenarios.sh

Done when

All scenarios pass on hardware, not just in simulation

3 Harden the product

What we do

Real energy meter and PWM pilot signal · grid code thresholds instead of placeholders · cryptographic billing check alongside the CRC

Done when

Tested and pass on real-world scenarios

4 Silicon

What we do

Tape out on Sky130 through an open shuttle run

Done when

First chips boot and pass the same scenarios

5 Pilot phase

What we do

Fit chips to a small fleet of chargers at one **partner site** · measure restart spread during real grid events

Done when

Staggered restart confirmed on a live grid

6 To market

What we do

Certify against charger, metering and grid standards · **sell the chip or license** the core to charger manufacturers

Done when

First manufacturer design win

Core, memory and both peripherals on one die.

Equipe 26

One CRC instruction.
Grid protection built in.

No network · No clock · No random numbers

7.0 MW → 0.88 MW

One chip, ready for the FPGA, ready for chargers.

ASEM

ChipInventor™